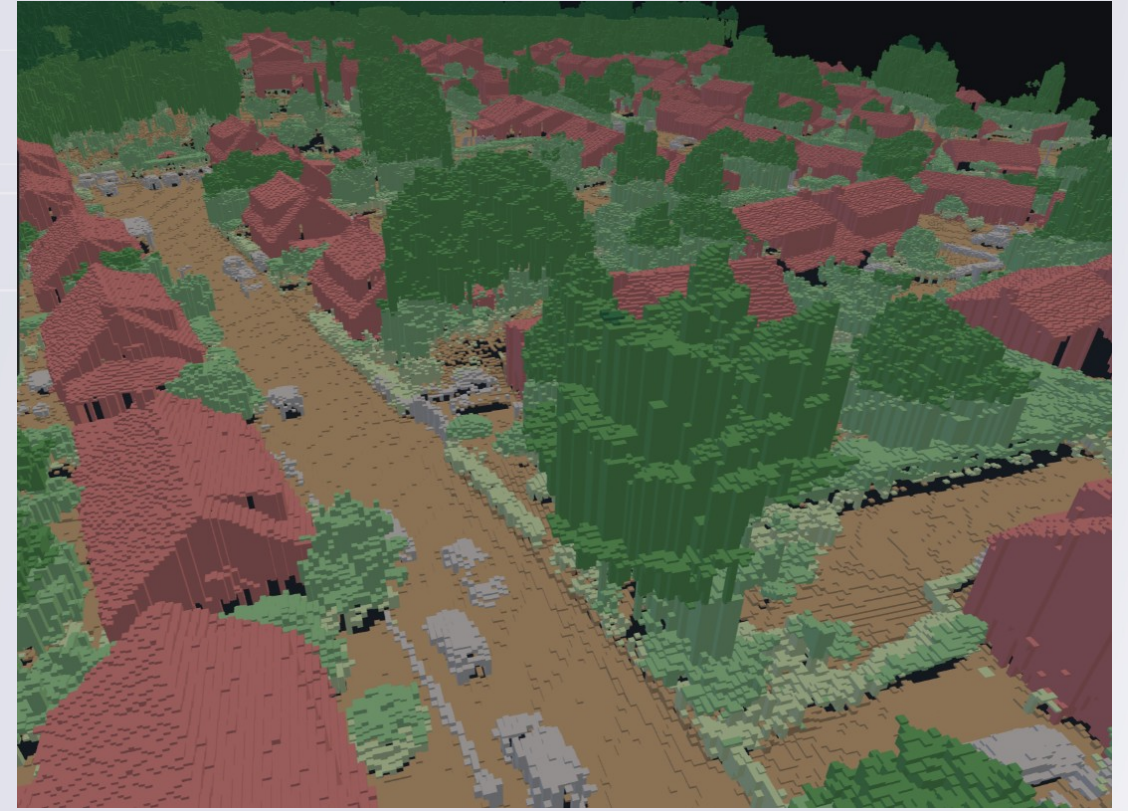




Voxelisation de données LiDAR urbaines

Une représentation 3-D sémantique de la ville, compressée en colonnes

Nikolaos Vynios - stage de M2, LIRIS / VCity

Encadrants : John Samuel, Gilles Gesquière,
Didier Puzenat

projet IA.rbre

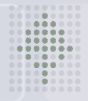
08/09/2026

Contexte : du laboratoire à IA.rbre

Le laboratoire :
LIRIS, projet VCity :
jumeaux numériques
de la ville

Le projet :
IA.rbre, avec TelesCoop
et la Métropole de Lyon

La question :
où planter des arbres
dans la métropole ?



ia • rbre

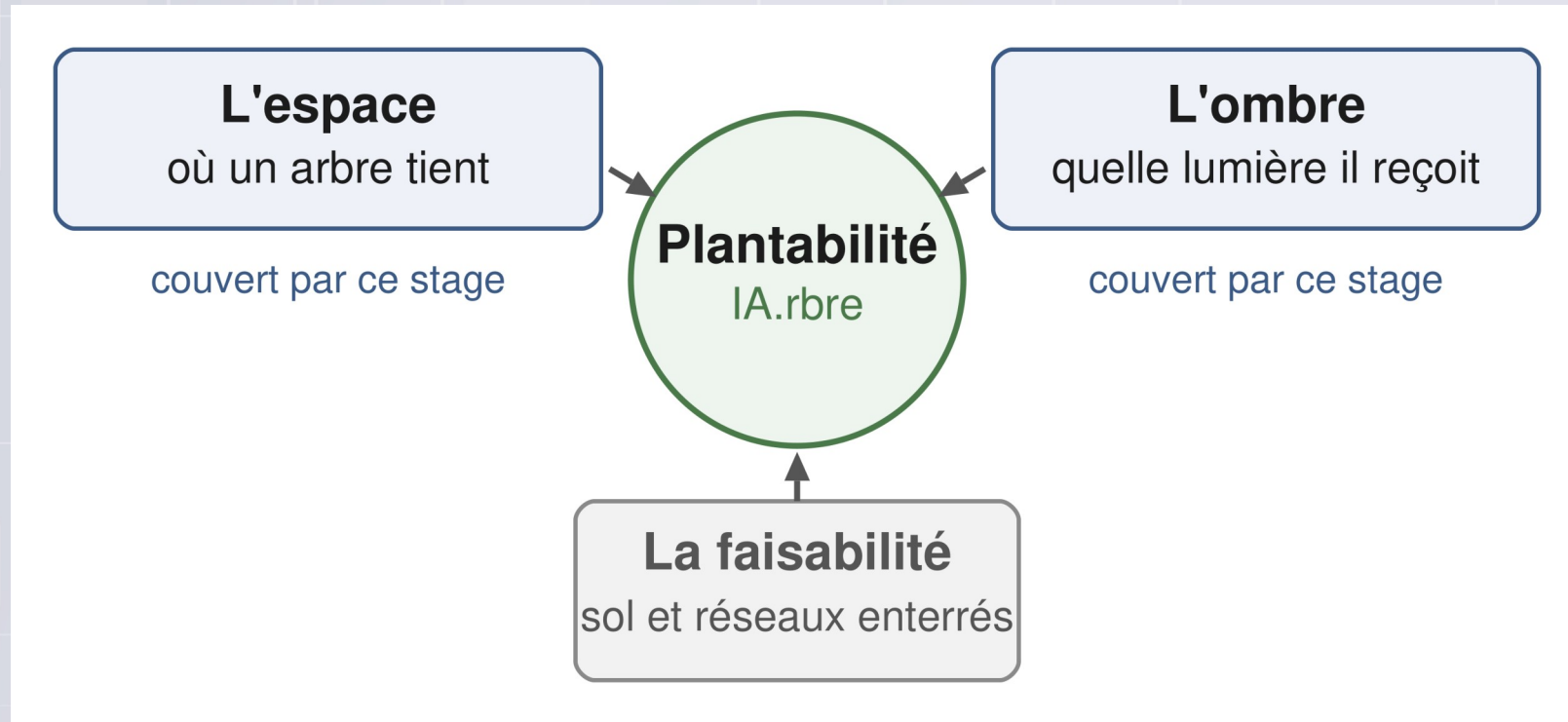


TelesCoop



un consortium mené par la coopérative TelesCoop avec des acteurs publics
financé partiellement par la Banque des Territoires

Contexte : la plantabilité

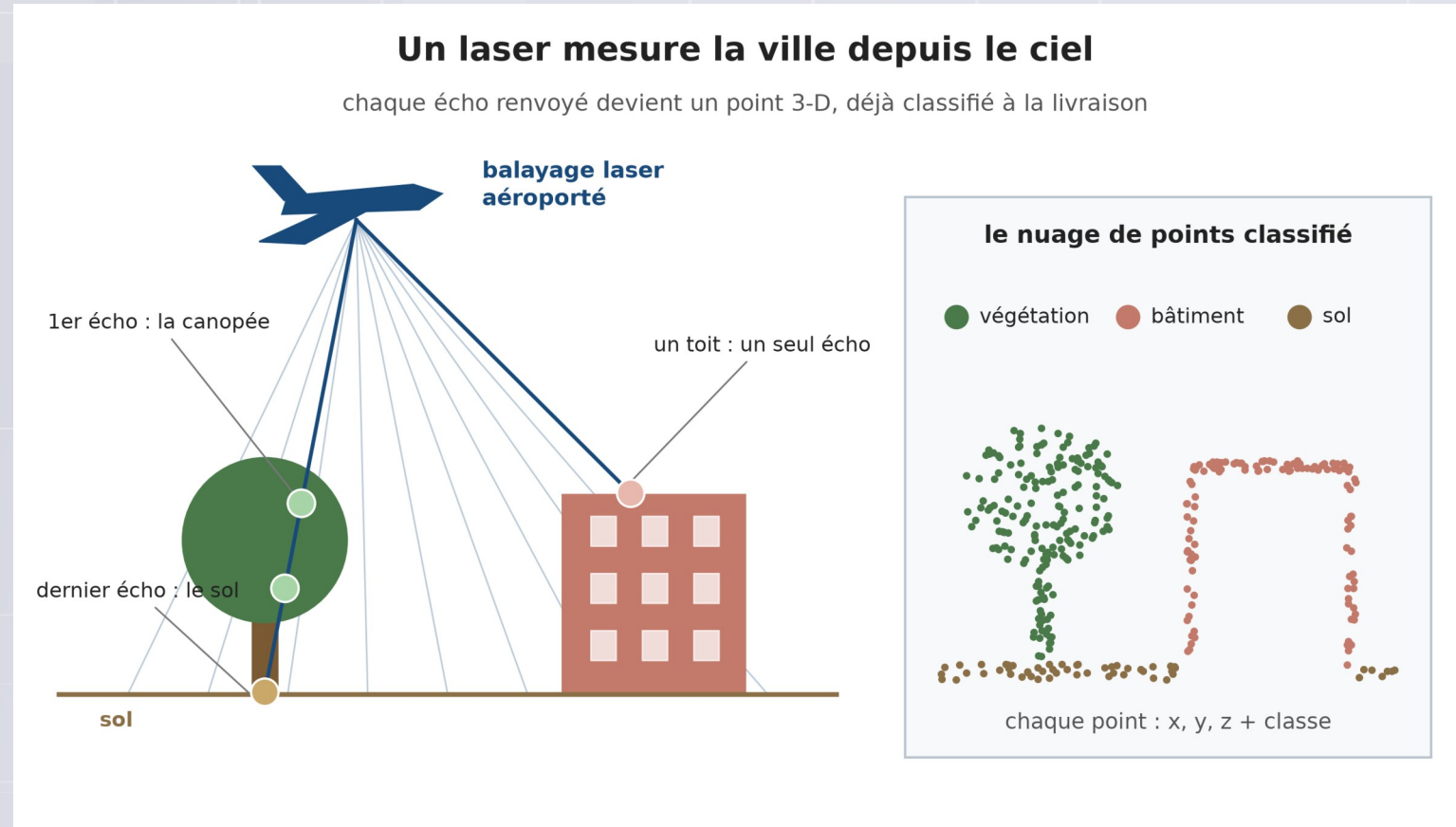


un relevé depuis le ciel ne voit pas le sous-sol : ce stage concerne l'espace et l'ombre

Trois objectifs : le voxeliseur

- 1** Concevoir un modèle voxel de la ville
végétation, sol, bâti, sous-sol
- 2** Le construire depuis les données existantes
LiDAR classifié de la Métropole de Lyon, 2 842 tuiles
- 3** Classifier et catégoriser les voxels
classes héritées, vide qualifié

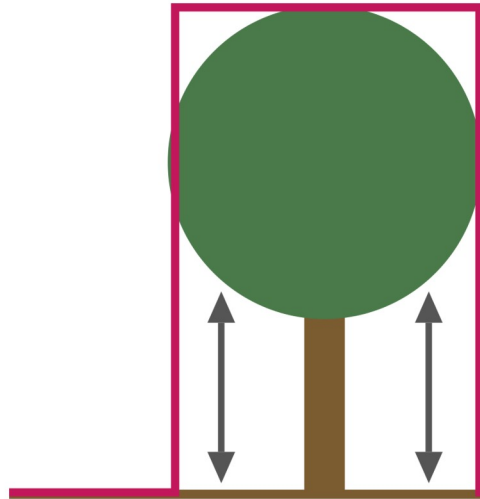
Les données : le LiDAR aérien



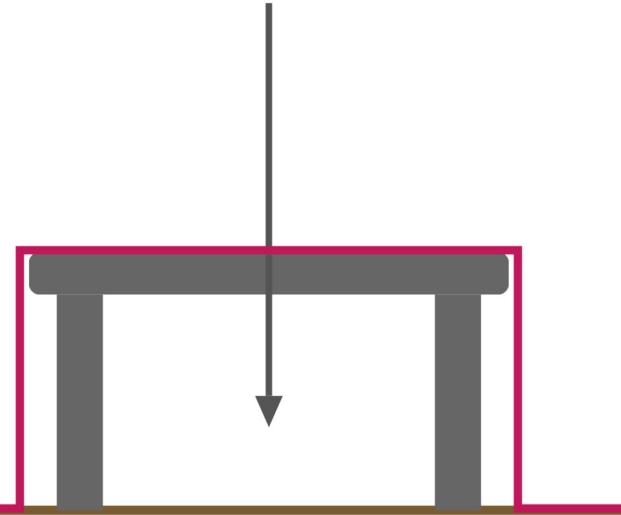
livraison LiDAR classifiée de la Métropole de Lyon, 2023 : chaque point est déjà étiqueté (LAS 1.2, TerraScan)

Pourquoi une surface (2D) ne suffit pas

de l'espace entre la couronne et le sol



une route sous un pont



le 2,5-D ne voit que ceci : une hauteur par cellule, rien en dessous

une hauteur par cellule : rien sous la couronne, rien sous le pont

Objectif 1 : concevoir le modèle

Trois objectifs : le voxeliseur

1

Concevoir un modèle voxel de la ville

végétation, sol, bâti, sous-sol

2

Le construire depuis les données existantes

LiDAR classifié de la Métropole de Lyon, 2 842 tuiles

3

Classifier et catégoriser les voxels

classes héritées, vide qualifié

Deux idées

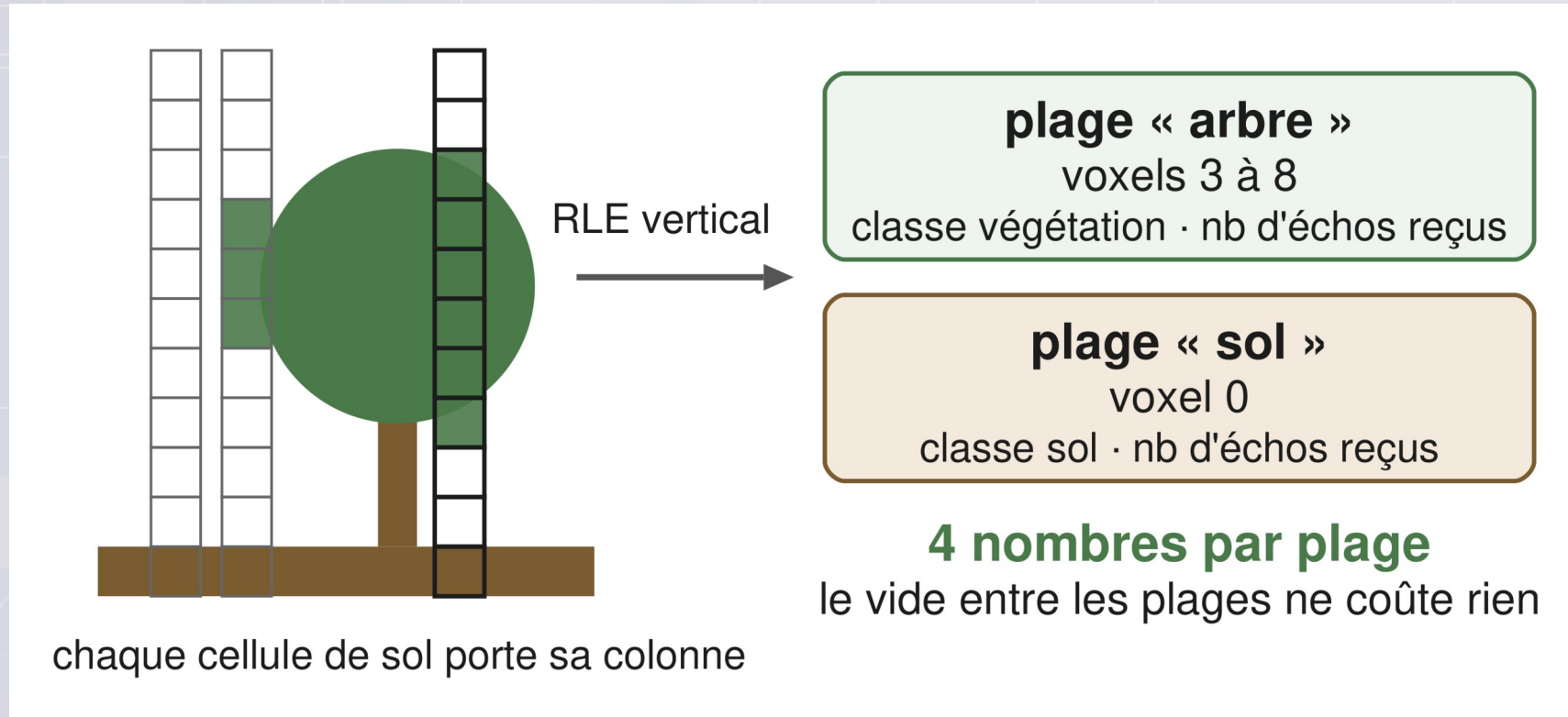
ce qu'une surface ne peut pas dire,
ce qu'une colonne peut dire

La question

répondre vite en vertical,
sans charger toute la zone

Notre approche : colonnes de voxels

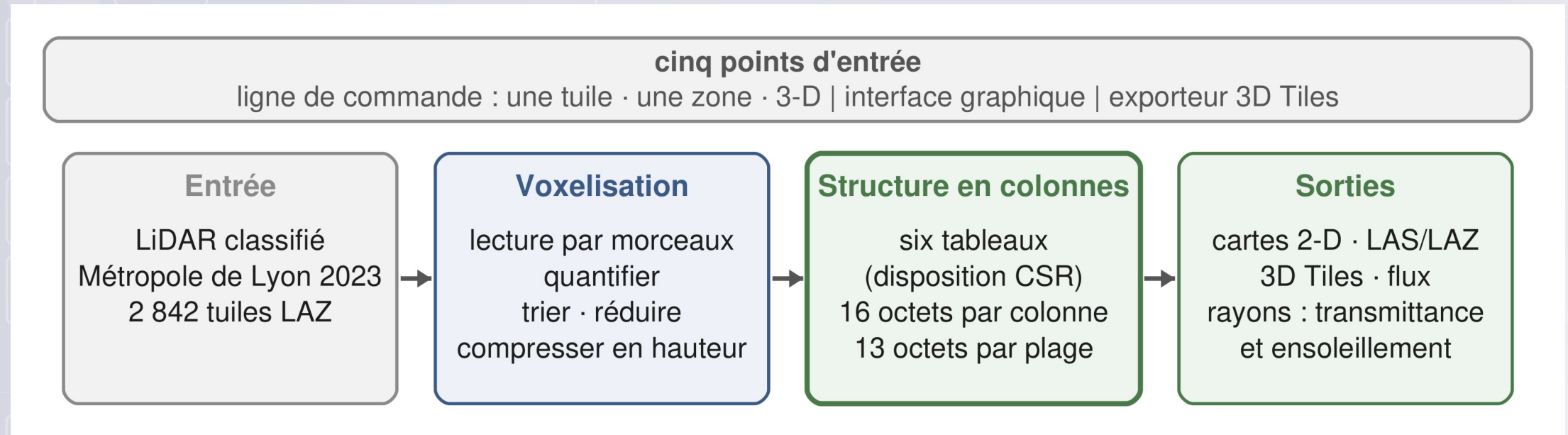
01



≈2 intervalles par colonne, quelle que soit la résolution

Notre approche : l'architecture, en un diagramme

O1



Fabriquer une colonne : lire et quantifier

O1

un point LiDAR

x, y, z en mètres · classe livrée

x 1831244.31 y 5177465.62 z 336.42
classe 2 (sol)

x 1831244.08 y 5177465.85 z 343.10
classe 4 (vég. moyenne)

x 1831244.44 y 5177465.71 z 351.87
classe 5 (vég. haute)



une cellule de grille

$ix = (x - x_0) / 0,5$; iy , iz de même

ix 488 iy 1931 iz 22
classe 2 · 1 écho

ix 488 iy 1931 iz 35
classe 4 · 1 écho

ix 488 iy 1931 iz 52
classe 5 · 1 écho

des entiers : la cellule est le voxel

carré E1 : 52 705 942 points lus, 3 995 579 colonnes occupées

Fabriquer une colonne : trier, réduire, compresser

O1

13 points, ordre d'arrivée

iz 7 · classe 4 · 1 écho
iz 2 · classe 2 · 2 échos
iz 13 · classe 5 · 1 écho
iz 3 · classe 2 · 1 écho
iz 0 · classe 2 · 3 échos
iz 14 · classe 5 · 2 échos
iz 1 · classe 2 · 1 écho
iz 8 · classe 4 · 2 échos
iz 12 · classe 5 · 1 écho
iz 3 · classe 2 · 2 échos
iz 6 · classe 4 · 1 écho
iz 14 · classe 5 · 1 écho
iz 7 · classe 4 · 1 écho

trier

par hauteur

iz14.2	iz14.1
iz 13 · 1	
iz 12 · 1	
iz 8 · 2	
iz7.1	iz7.1
iz 6 · 1	
iz3.1	iz3.2
iz 2 · 2	
iz 1 · 1	
iz 0 · 3	

réduire

une entrée par cellule

iz 14 · 3
iz 13 · 1
iz 12 · 1
iz 8 · 2
iz 7 · 2
iz 6 · 1
iz 3 · 3
iz 2 · 2
iz 1 · 1
iz 0 · 3

compresser

plages verticales

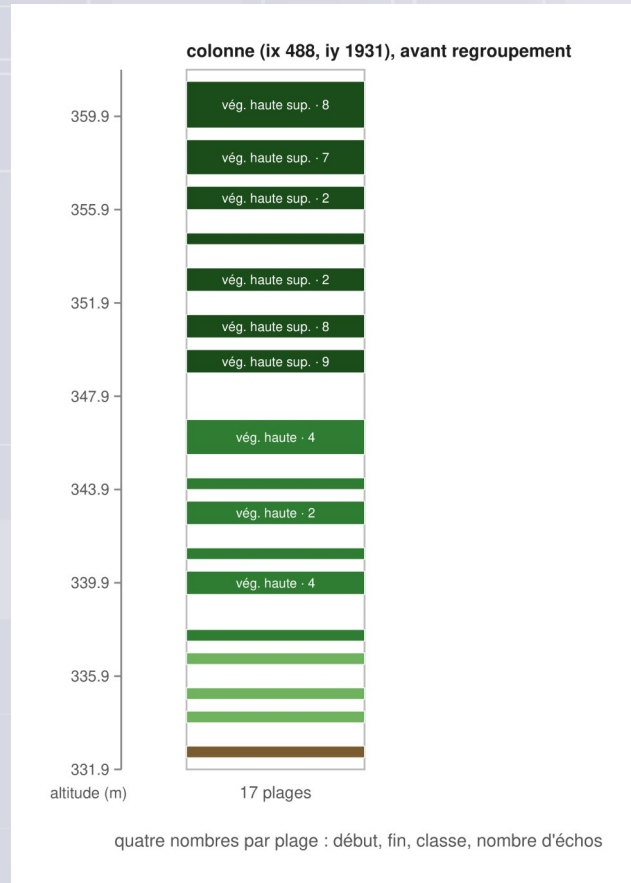
[12, 15) vég. haute 5 échos
[6, 9) vég. moyenne 5 échos
[0, 4) sol 9 échos

13 points → 10 cellules → 3 plages ; le vide n'est pas stocké

un arbre de trente mètres se transforme en deux - quatre plages au lieu de milliers de points

Une colonne réelle

O1



Colonne (ix 488, iy 1931) du carré E1 :
17 plages avant regroupement,
62 échos, de 332,4 à 361,4 m d'altitude ;
sol, végétation moyenne, haute, haute supérieure

chaque plage se lit seule : la colonne n'a besoin d'aucune voisine

six tableaux, disposition CSR

par colonne : 16 octets

keys (uint64) : la clé de la colonne,
ix et iy dans ses deux moitiés
off (int64) : où commencent ses plages
colonne 977 169 : plages 1 673 381 à 1 673 384

par plage : 13 octets

zs, ze (int32) : début, fin exclue
cl (uint8) : la classe
ct (int32) : le nombre d'échos
en-tête : origine, taille de cellule

la colonne (ix 488, iy 1931) après regroupement

zs	ze	classe	échos	altitude
14	15	2 (sol)	8	332,4 à 332,9 m
17	23	4 (vég. moyenne)	3	333,9 à 336,9 m
24	43	5 (vég. haute)	14	337,4 à 346,9 m
47	72	8 (vég. haute sup.)	37	348,9 à 361,4 m

le carré E1 entier tient dans 140 Mo : 3 995 579 colonnes et 5 830 031 plages

Objectif 2 : construire à l'échelle

Trois objectifs : le voxeliseur

- 1 Concevoir un modèle voxel de la ville
végétation, sol, bâti, sous-sol
- 2 **Le construire depuis les données existantes**
LiDAR classifié de la Métropole de Lyon, 2 842 tuiles
- 3 Classifier et catégoriser les voxels
classes héritées, vide qualifié

Un problème : l'échelle

deux faces :
la mémoire, le temps

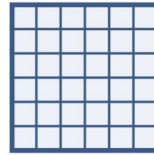
L'ordre de grandeur

52 milliards de points ;
aucune station ne les tient

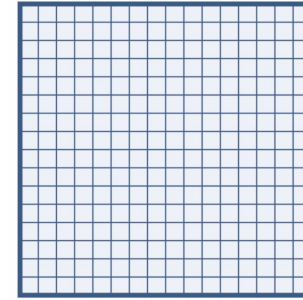
Le problème n° 1 : l'échelle, d'une tuile à la Métropole



1 tuile
28 M points



36 tuiles
752 M points



2 842 tuiles
51,9 Md points

chargée naïvement :
~1,5 To de RAM d'un coup



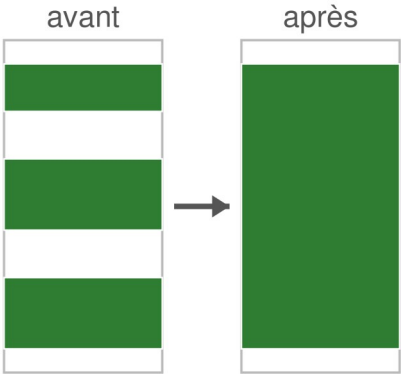
structure de tableaux
+ mémoire bornée :
pic ~1,4 Go

fusion de la Métropole : 2 h → 2 min 30

→ un algorithme plus performant plutôt qu'une machine plus grosse

Regrouper les plages : la règle et l'effet

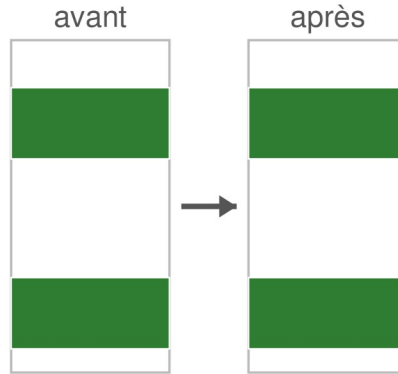
écart $\leq 1,5$ m, même classe



regroupées : une plage

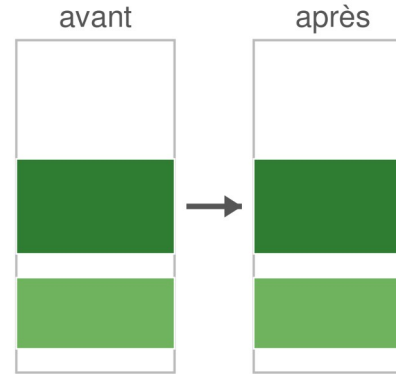
tolérance en mètres : la même décision à toute résolution ; aucune classe ne change

écart $> 1,5$ m



gardées séparées

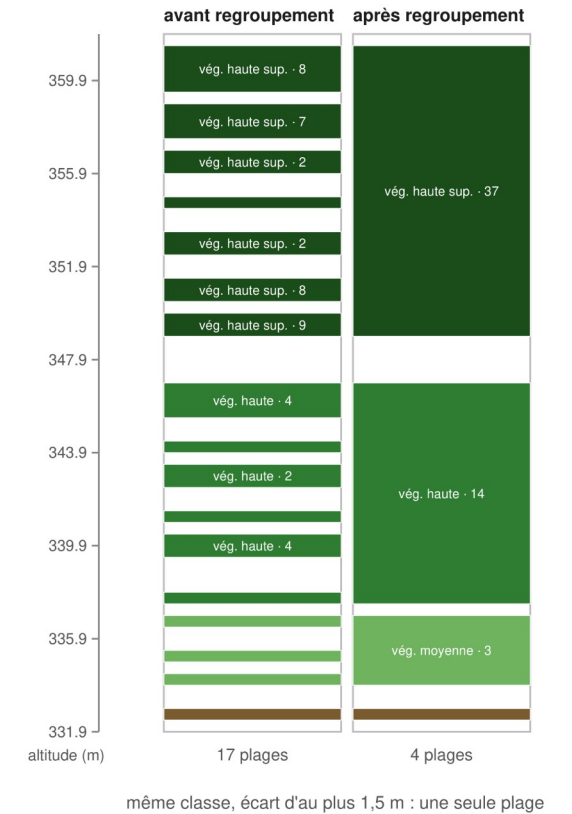
classes différentes



jamais fusionnées

Sur le carré E1 :

6 630 970 plages avant, 5 830 031 après : 12,1 % de moins ;
458 583 colonnes touchées sur 3 995 579 (11,5 %)



1,5 m en production ; le stockage brut reste disponible pour les analyses plus fines des vides

Tuile par tuile : le principe



à l'échelle de la Métropole : 2 842 tuiles, 2 842 fragments, une fusion par bandes
chaque sortie dans son propre processus : une panne ne coûte qu'un produit

carré E1 : quatre fragments de 5 à 7 Mo sur disque, une structure de 140 Mo

Objectif 3 : classifier les voxels

Trois objectifs : le voxeliseur

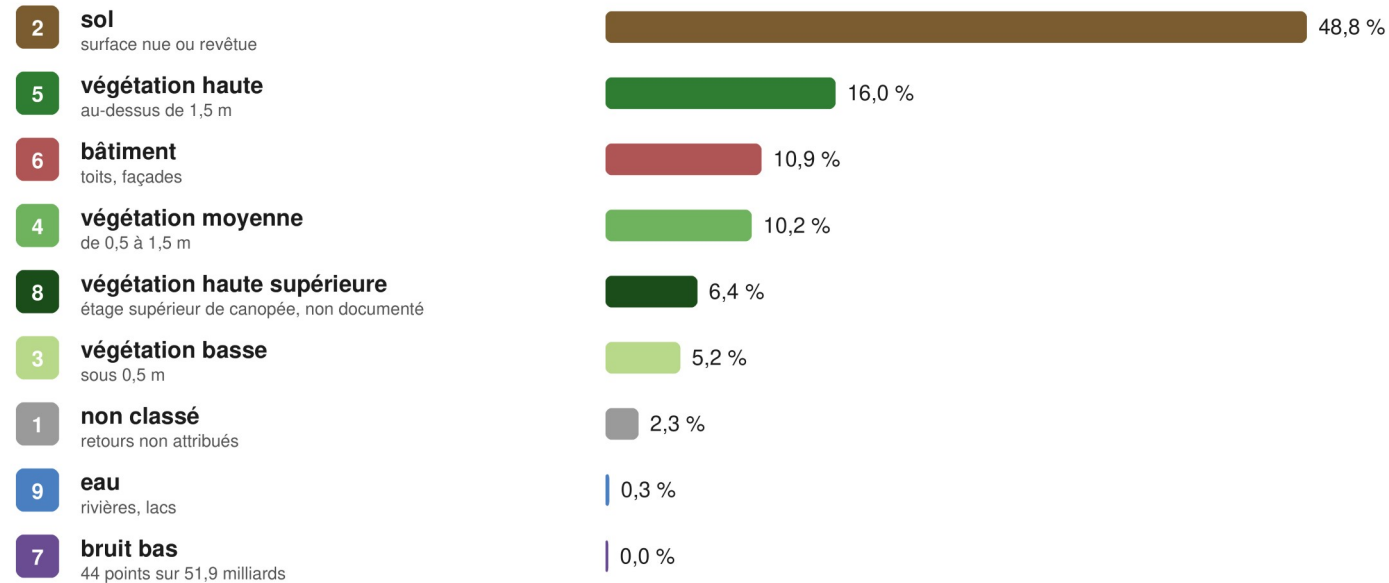
- 1 Concevoir un modèle voxel de la ville
végétation, sol, bâti, sous-sol
- 2 Le construire depuis les données existantes
LiDAR classifié de la Métropole de Lyon, 2 842 tuiles
- 3 **Classifier et catégoriser les voxels**
classes héritées, vide qualifié

Une rencontre avec le réel
une classe que rien
ne documente

Le principe
chaque voxel porte sa nature,
classe par classe, séparément

neuf codes ASPRS, mesurés sur les 2 842 tuiles

51 912 212 206 points ; chaque point porte un code de classe attribué avant nous



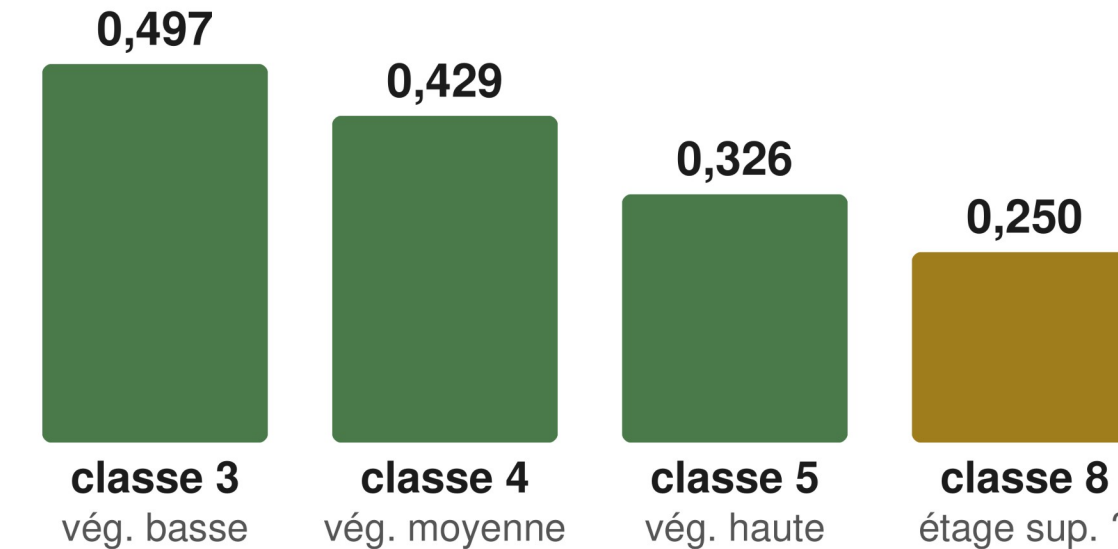
codes 1 à 9 de la norme LAS 1.2 ; aucun code étendu (64 à 67) dans la livraison

la norme réserve le code 8 ; la livraison l'emploie sur 2 691 tuiles sur 2 842 : à élucider

Le problème n° 2 : la classe 8

La classe 8 se comporte comme de la végétation

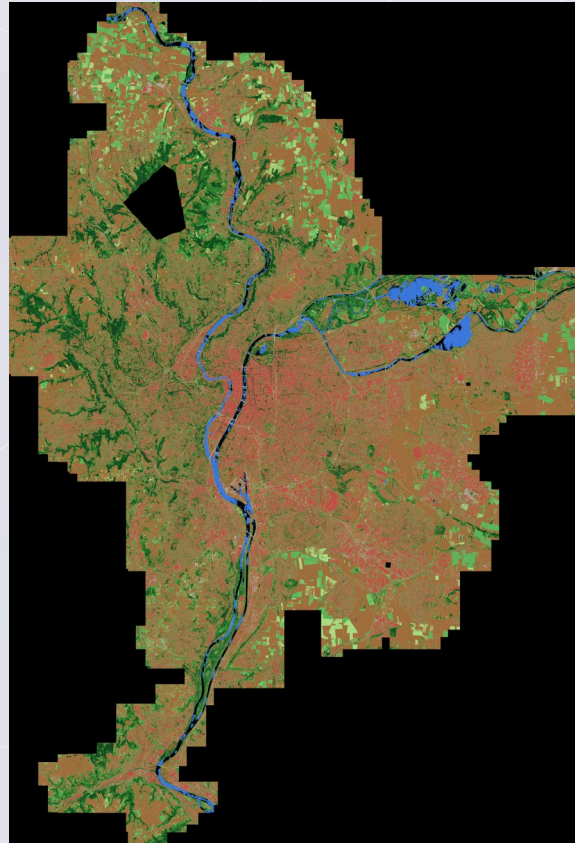
coefficient d'extinction : à quel point un mètre de matière atténue le faisceau
(mesuré sur le corpus, en 1/m, cellules de 0,5 m)



code absent des notes de livraison : enquête, pas filtrage

nommée étage supérieur de canopée : 6,4 % du corpus, gardée séparée dans chaque résultat

51,9 milliards de points
voxélisés de bout en bout



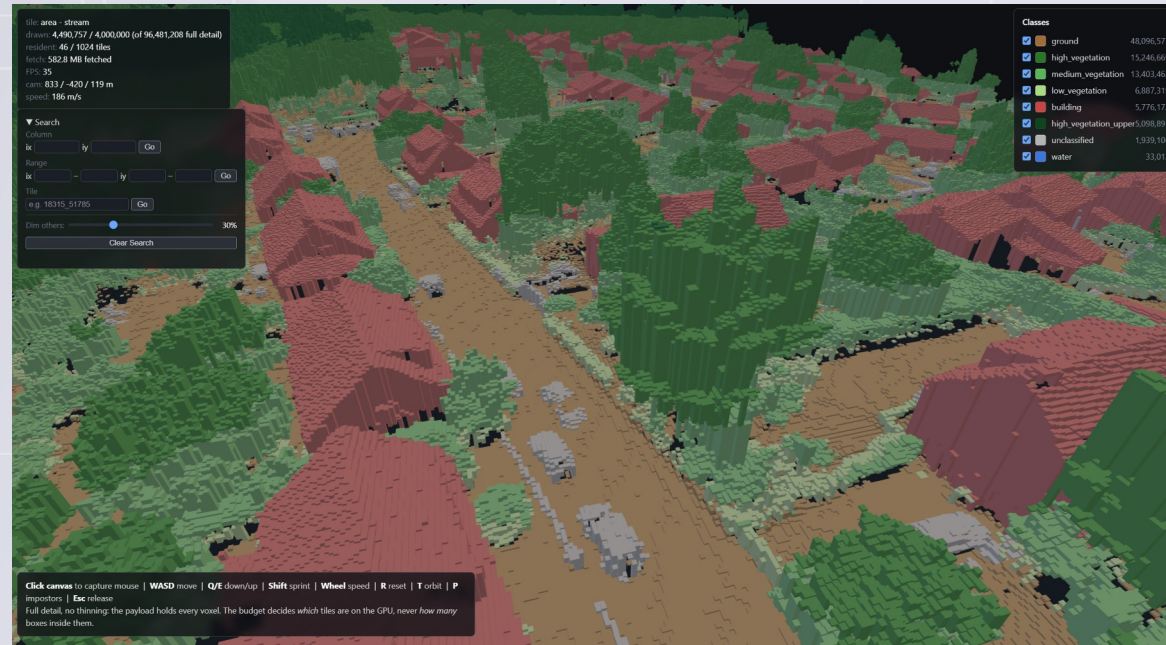
facteur 8,3
sur le LAZ déjà compressé

facteur 8,3 sur le LAZ livré (3 tuiles, grille 1 m) ; corpus : 2 842 tuiles, 3 résolutions
rendu des classes de la Métropole entière (0,5 m cube), classe par cellule de 0,5 m × 0,5 m

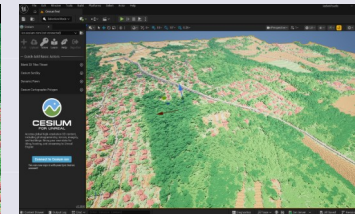
Les sorties : LAS/LAZ, flux three.js et 3D Tiles

02

Valideur officiel :
zéro erreur ; deux
réserves du valideur ;
défaut masqué corrigé

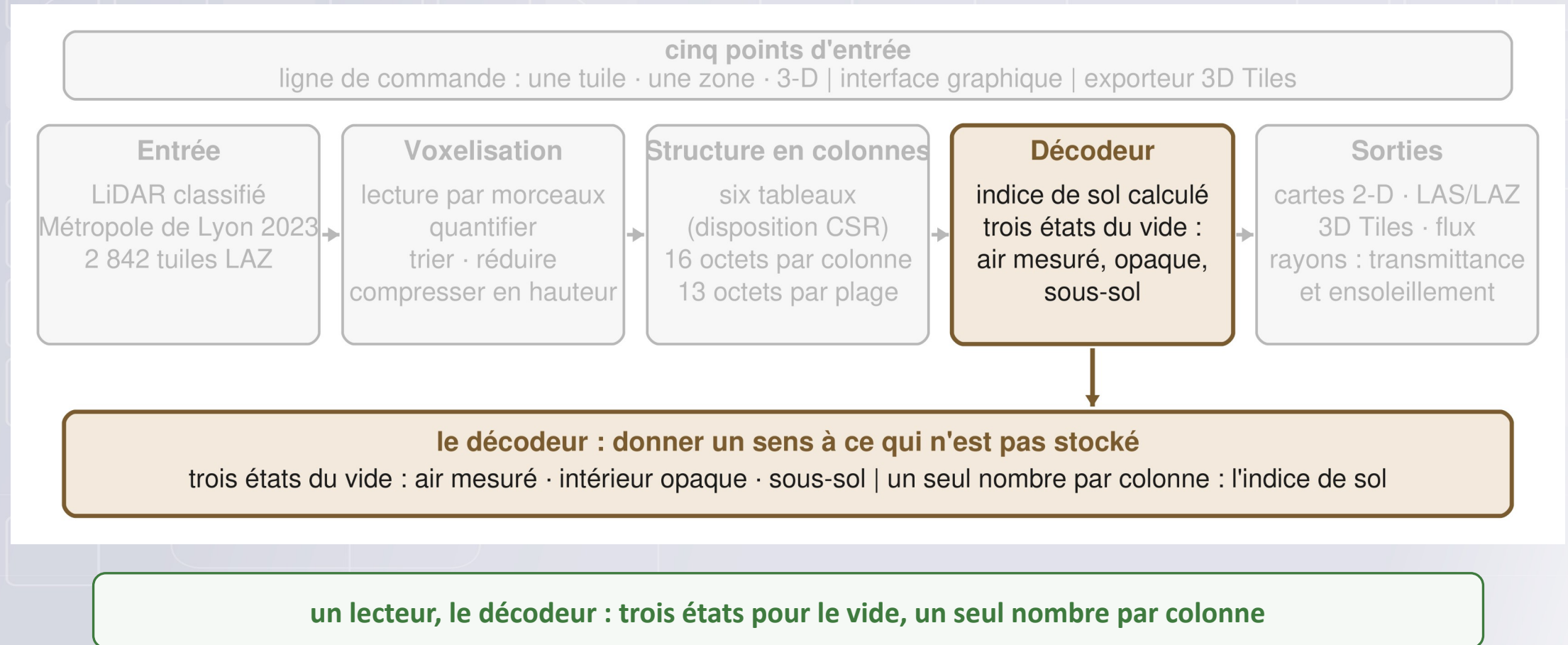


CesiumJS - iTowns
Unreal Engine
projection locale :
RGF93 / CC46 (EPSG:3946)

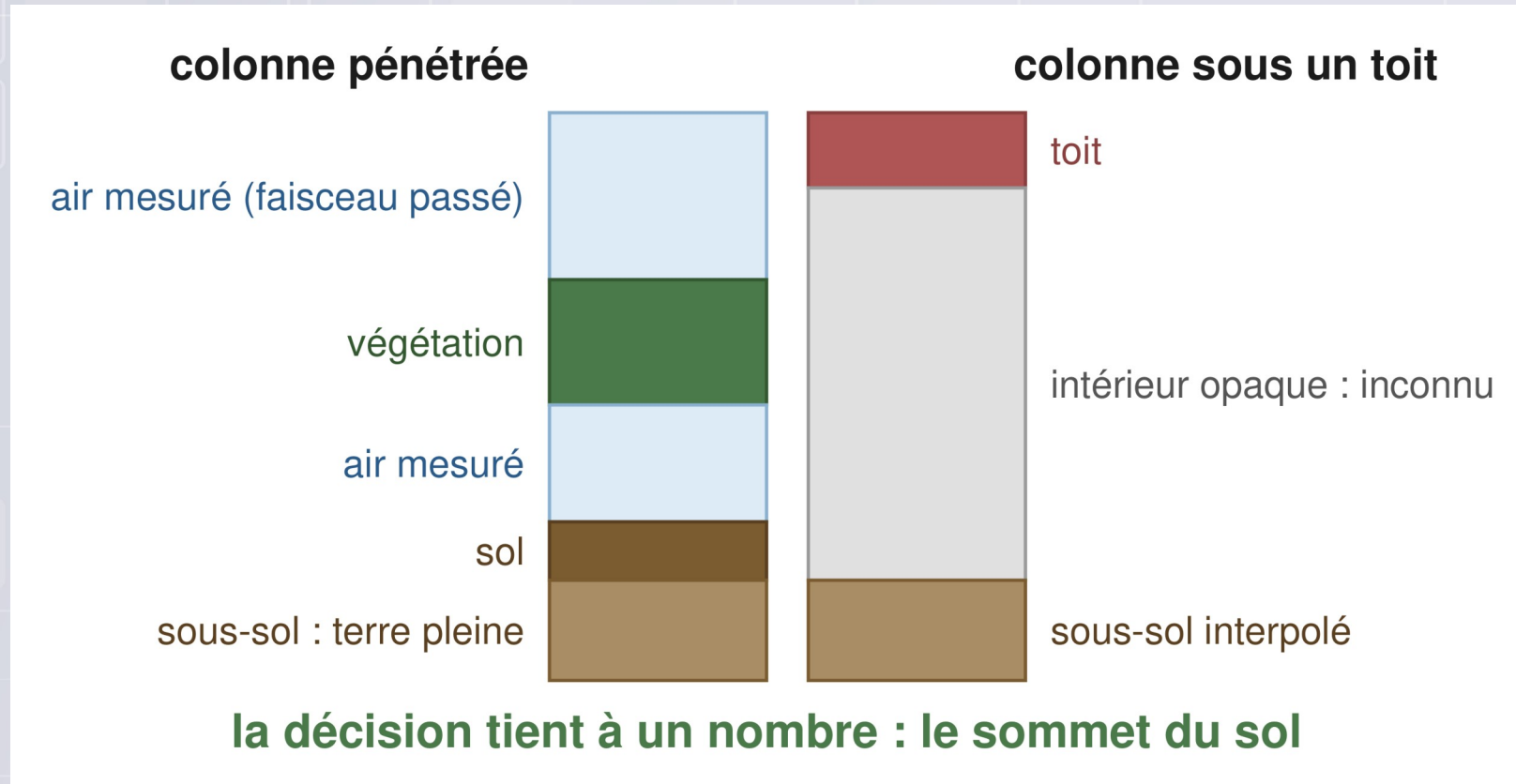


un seul artefact au centre ; LAS/LAZ, flux three.js et 3D Tiles 1.1 en sortie

Le problème n° 3 : ce que la structure ne stocke pas, le vide

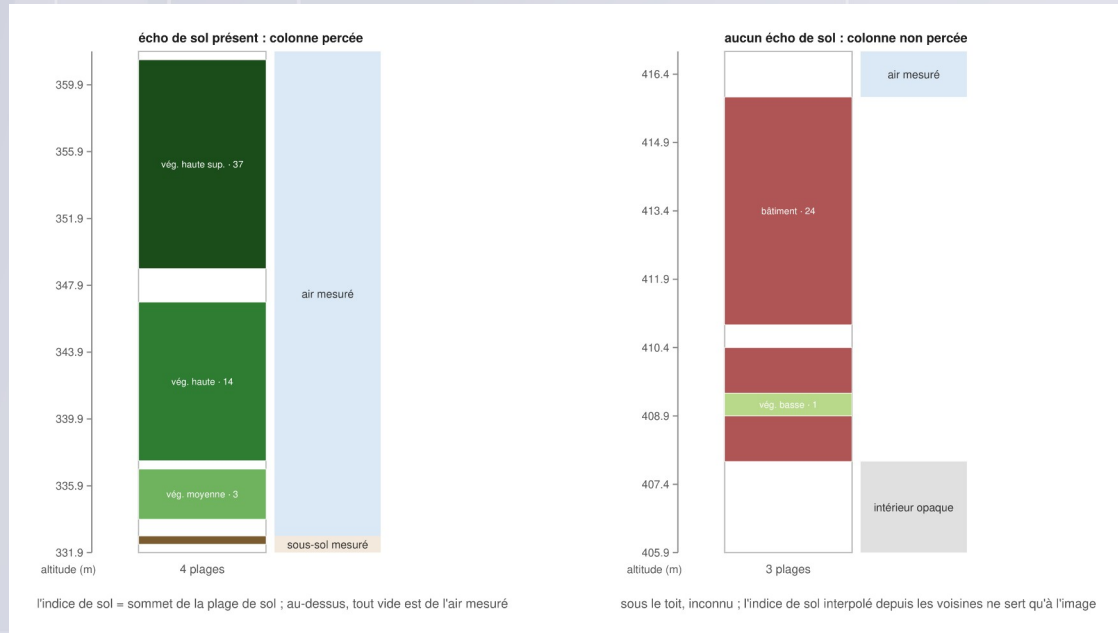


Notre approche : trois formes de vide



confondre les vides fausse tout calcul physique en aval

Le sous-sol : mesuré ou interpolé



Deux règles :

- un écho de sol : colonne percée ; terre en dessous, air mesuré au-dessus ;
- aucun écho de sol : sous le toit, intérieur opaque, traité comme plein ;

l'indice interpolé depuis les voisines ne sert qu'aux images

un seul nombre par colonne, l'indice de sol, qualifie tout le vide

Limites

le LiDAR ne voit pas le sous-sol :
espace et ombre, pas la faisabilité

faisceau supposé vertical (nadiral) :
atténué, résidu consigné

classification livrée : une entrée non réévaluée ;
ses erreurs se propagent à la structure

le regroupement comble les vrais vides :
les analyses fidèles aux vides lisent le brut

Conclusion

Livré :
la structure en colonnes ;
la Métropole entière, trois résolutions

Livré :
classification héritée, classe 8 tranchée,
trois formes de vide

Livré :
tuile par tuile, mémoire bornée,
résultat identique à la construction directe

Reproductible :
code sur GitHub (VCityTeam/UD-IArbre-Research),
de nombreux tests, image Docker, documentation

trois objectifs tenus : concevoir, construire à l'échelle, classifier

Perspectives

Post-traitements potentiels :

l'ombrage volumique par lancer de rayons ;
la comparaison au 2,5-D (premiers résultats en annexe)

Données :

le sous-sol (réseaux, sols) pour la faisabilité ;
le corpus complet à 0,25 m

Structure et classes :

des classes plus fines, une couche indexée par cellule ;
une tolérance de regroupement par classe

Temps et intégration :

deux campagnes LiDAR : carte de changement par colonne ;
les tilesets servis par la plateforme IA.rbre

l'objectif reste la plantabilité : la faisabilité passera par le sous-sol

Bibliographie : les sources principales

Données et formats

IGN, « LiDAR HD : vers une nouvelle cartographie 3D du territoire », 2023

Métropole de Lyon, « Data Grand Lyon », portail open data, 2023

OGC, 3D Tiles Specification 1.1, doc. 22-025r4, 2023

Structures voxel

Aleksandrov, Zlatanova, Heslop, « Voxelisation algorithms and data structures: a review », Sensors, 2021

Museth, « VDB: high-resolution sparse volumes with dynamic topology », ACM Trans. Graph., 2013

Peytavie, Galin, Grosjean, Mérillou, « Arches: a framework for modeling complex terrains », Comput. Graph. Forum, 2009

Encodage par plages

Tzamarias, Chow, Blanes, Serra-Sagristà, « Fast run-length compression of point cloud geometry », IEEE Trans. Image Process., 2022

Golomb, « Run-length encodings », IEEE Trans. Inf. Theory, 1966

Abadi et al., « The design and implementation of modern column-oriented database systems », Found. Trends Databases, 2013

Lumière et soleil

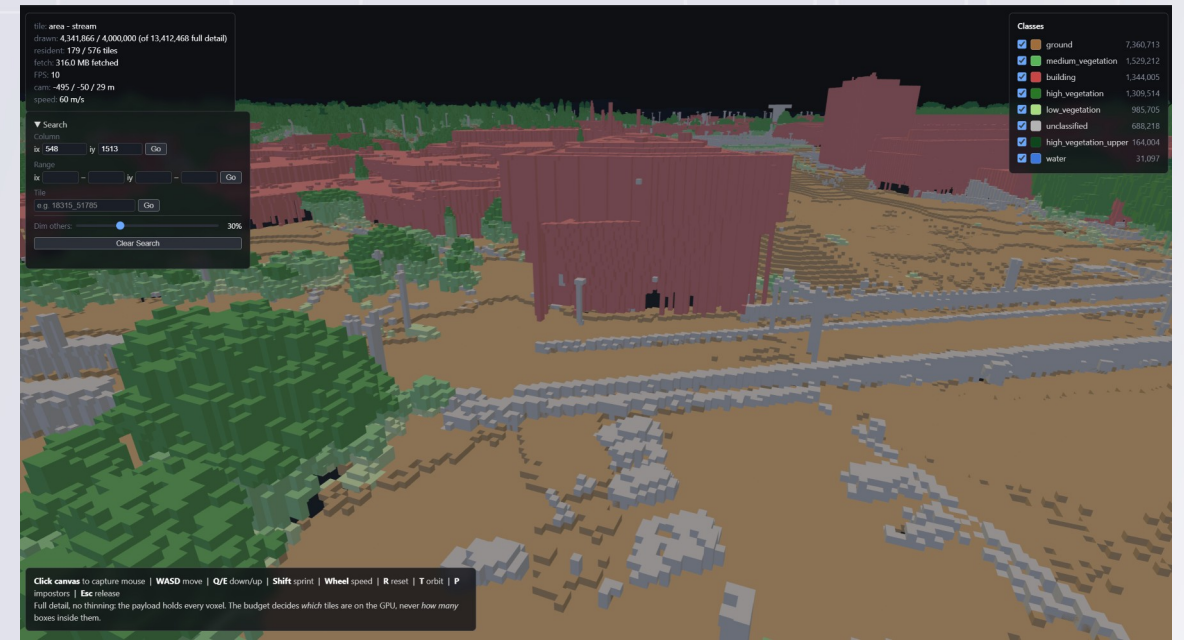
Beer, « Bestimmung der Absorption des rothen Lichts in farbigen Flüssigkeiten », Ann. Phys. Chem., 1852

Monsi, Saeki, « Über den Lichtfaktor in den Pflanzengesellschaften... », Jpn. J. Bot., 1953

NOAA, « Solar Calculator », 2024 ; Meeus, Astronomical Algorithms, 1998

VCity Team, pySunlight, GitHub, 2023

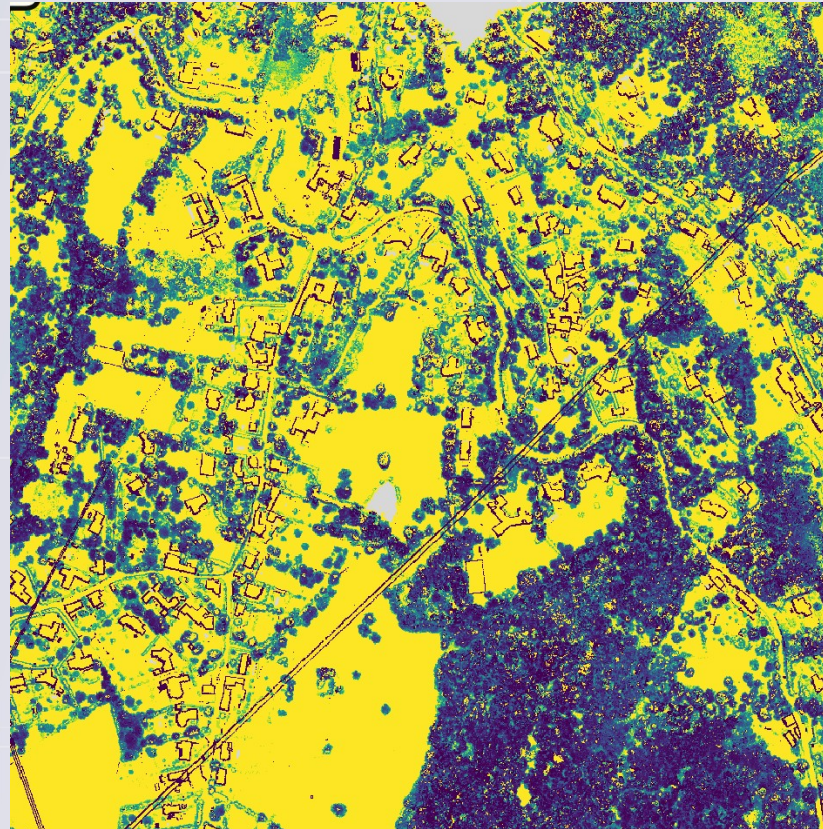
46 références citées dans le rapport, sur 118 sources consultées



Merci de votre attention - des questions ?

Annexe : la lumière à travers la canopée

opaque :
une feuille arrête tout



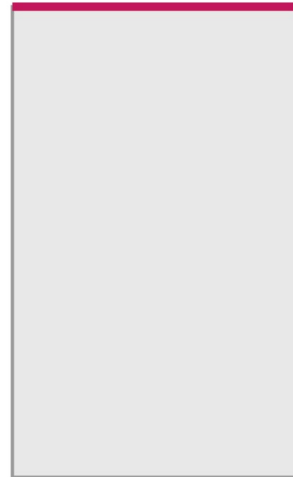
sensible aux classes :
+1,8 h de soleil direct
en zone végétalisée

zone B04, rayons verticaux : transmittance par classe, du bleu foncé (opaque) au jaune (transparent) ; gris : sans donnée

Annexe : ce que le 2,5-D ne peut pas exprimer

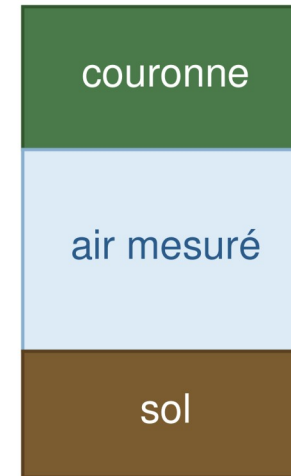
Un champ de hauteurs n'a pas de dessous

modèle de surface (2,5-D)



un seul corps, depuis le sommet
le rayon s'arrête au sommet de la couronne

structure en colonnes

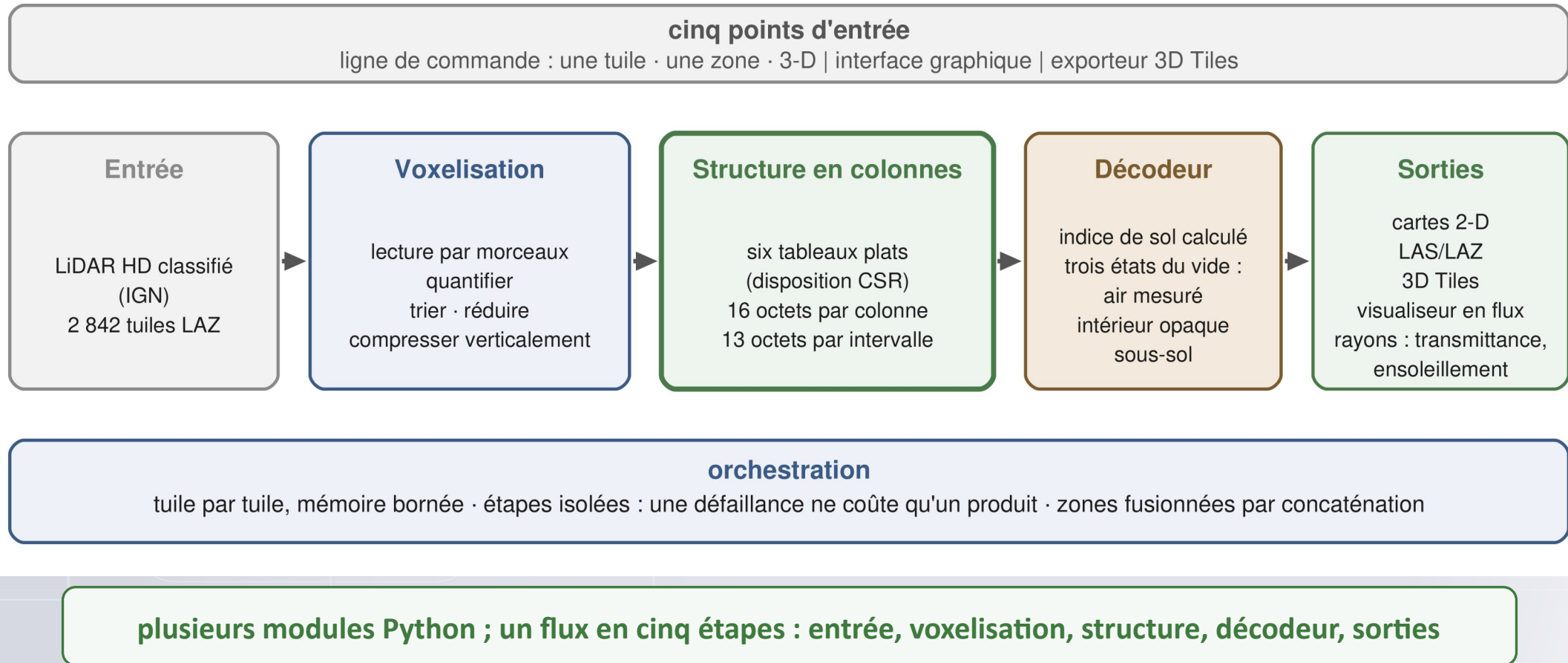


deux corps, séparés par de l'air mesuré
la lumière sous la couronne existe

9,7 à 11,7 h de soleil en trop par jour sous la canopée

Annexe : l'architecture complète

O1



Annexe : le campus Porte des Alpes, voxelis 

16 tuiles LiDAR
236 millions de points
classe dominante
par cellule de 25 cm



35,9 M de colonnes
47,3 M d'intervalles
au niveau le plus fin

1,5 km de c t  : le campus entier et le couloir T2 Rebufer - Porte des Alpes

visualiseur three.js :
budget GPU explicite,
35 images/s mesurées

CesiumJS en second :
seuil 48 sûr jusqu'à 1080 px,
sinon 64 / 96

iTowns :
survol seulement

Unreal Engine :
captures, jamais en direct

visualiseur en flux (le plus sûr) puis CesiumJS après contrôle de la hauteur d'écran

Annexe : la Métropole en chiffres

2,0 m : 8,1 Go
2,60 intervalles par colonne

1,0 m : 27,9 Go
2,08 intervalles par colonne

0,5 m : 101 Go
1,78 intervalle par colonne

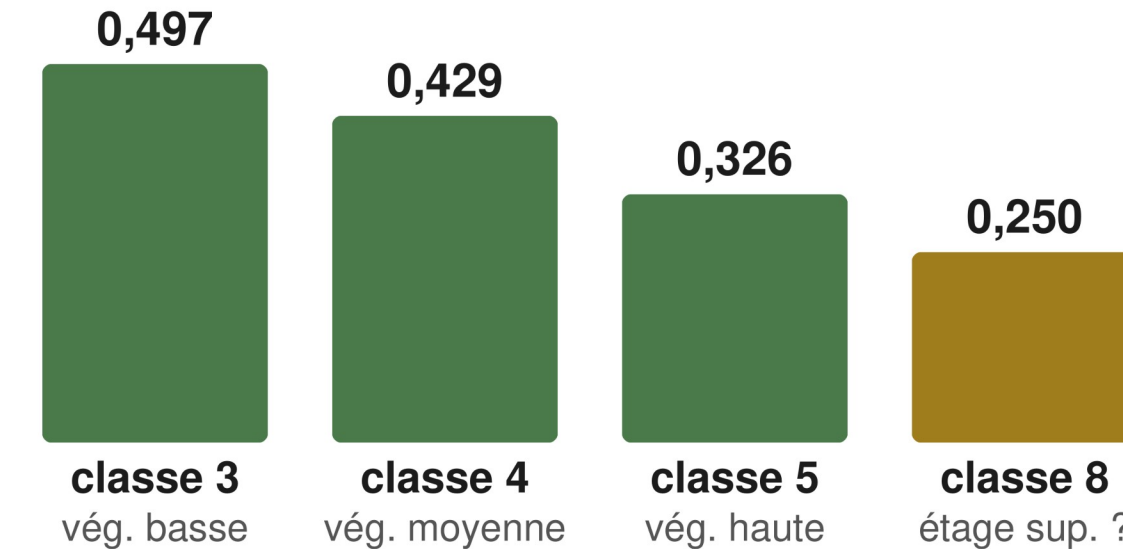
fusion complète de la Métropole :
environ 2 min 30

2 842 tuiles - 51,9 milliards de points - trois résolutions regroupées

Annexe : la classe 8 en détail

La classe 8 se comporte comme de la végétation

coefficient d'extinction : à quel point un mètre de matière atténue le faisceau
(mesuré sur le corpus, en 1/m, cellules de 0,5 m)



code absent des notes de livraison : enquête, pas filtrage

k mesuré : 0,497 - 0,429 - 0,326 - 0,250 (1/m, cellules de 0,5 m)

Annexe : chiffres de réserve

48 modules Python
~27 000 lignes (voxeliseur)

plusieurs centaines de tests
~16 000 lignes de tests

estimateur : fiable à 1 m,
biais consigné ailleurs

Python, numpy, three.js,
3D Tiles, Docker

fiche d'identité du code

Quatre tuiles de 500 m :

18310_51765, 18310_51770,
18315_51765, 18315_51770

Grille 0,5 m :

3 995 579 colonnes occupées sur 4 000 000 ;
52 705 942 points

Plages :

6 630 970 avant regroupement,
5 830 031 après ; 1,46 par colonne

Direct contre partitionné :

six tableaux égaux élément par élément ;
88 s contre 95 s

campagne du 8 août 2026, expérience E1, manifeste et résultats archivés

Annexe : E8, les quarante cellules

Dix zones de 2 km :
tirées au sort (graine 20260808),
séparées d'au moins 3 km

Quatre grilles :
1 m, 0,5 m, 0,25 m,
0,25 m en plan et 0,1 m en hauteur

À 0,5 m :
151,5 M colonnes, 269,1 M plages,
1,78 par colonne, 49,9 min pour les dix

Cinq portes exactes par cellule :
points, colonnes, plages, octets, manifeste ;
40 sur 40 au premier essai

diviser la cellule par deux coûte 21 % de temps en plus : le travail est par point, pas par cellule

Annexe : E5, l'estimateur contre la mesure

2 m (Run1, Run2) :

prédit 11,9 Go, mesuré 18,3 et 18,7 Go
rapport 1,53 et 1,57

1 m (Run3, Run4) :

prédit 28,0 Go, mesuré 26,6 et 27,1 Go
rapport 0,95 et 0,97

0,5 m (Run5, Run6) :

prédit 92,5 Go, mesuré 25,1 et 24,6 Go
rapport 0,27

Deux lecteurs indépendants :

même pic, même nombre de lignes,
sur les six journaux de ressources

l'estimateur ne voit que la surface au sol : fiable à 1 m, biais consigné ailleurs

Annexe : les encodeurs par résolution

v1 lissage des classes :

plages en moins 14 à 29 % (1 m), 6 à 13 % (0,5 m),
1 à 3 % (0,25 m) ; J végétation 0,87 à 0,99

v2 comblement des écarts ≤ 1 m :

10 à 20 % (1 m), 4 à 8 % (0,5 m), 1 à 2 % (0,25 m) ;
J = 1,00 partout

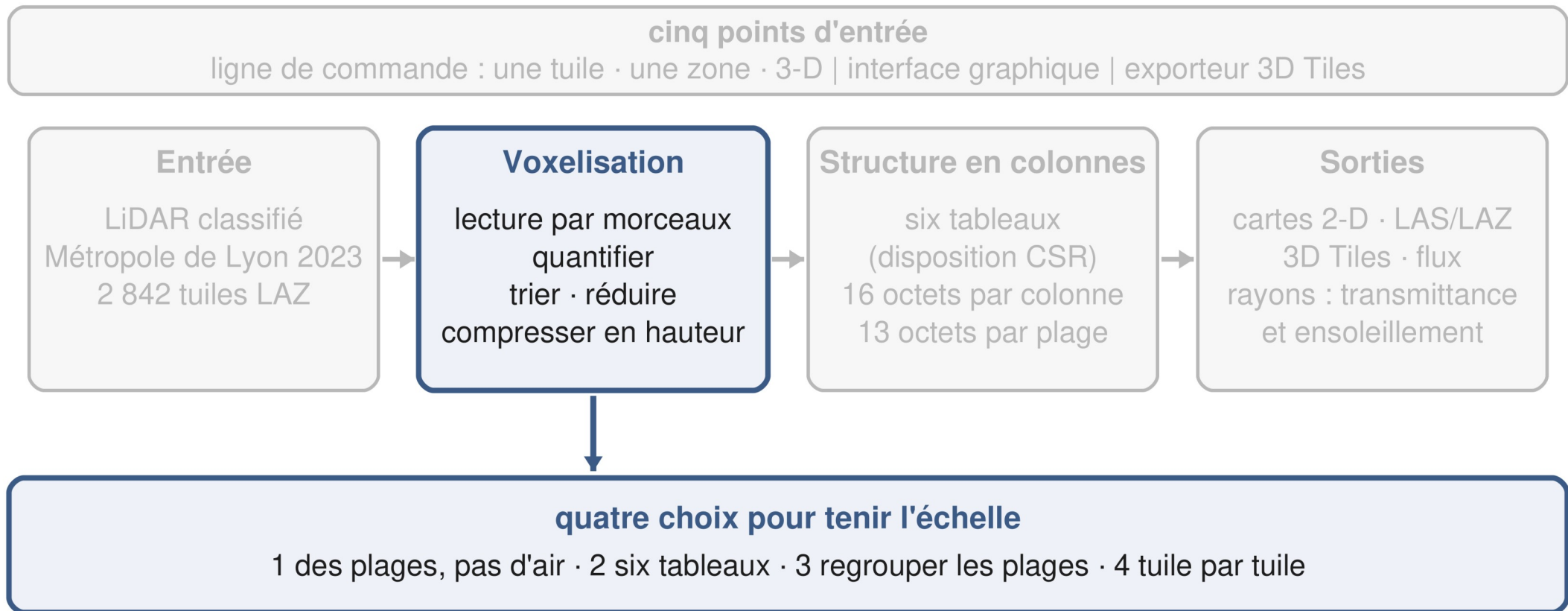
v3 classes mineures :

0 à 4 % (1 m), ≈ 0 en dessous ;
J = 0,97 à 1,00

v4 majorité au-dessus de 5 m :

-2 à 5 % ; J végétation 0,56 à 0,78 (1 m),
0,68 à 0,91 (0,5 m) ; rejeté

dix carrés d'un kilomètre, quatre résolutions, cinq encodeurs : 200 enregistrements



Annexe : les quatre choix, en détail

annexe

Des plages, pas de l'air :

un arbre de trente mètres = deux - quatre plages ;
10 à 40 fois moins d'éléments que de points

Six tableaux plats :

16 octets par colonne, 13 par plage ;
16 fois moins qu'un objet par colonne

Tuile par tuile :

un fragment à la fois, fusion en une seule passe ;
2 842 fragments : 2 h → 2 min 30

Estimateur et étapes isolées :

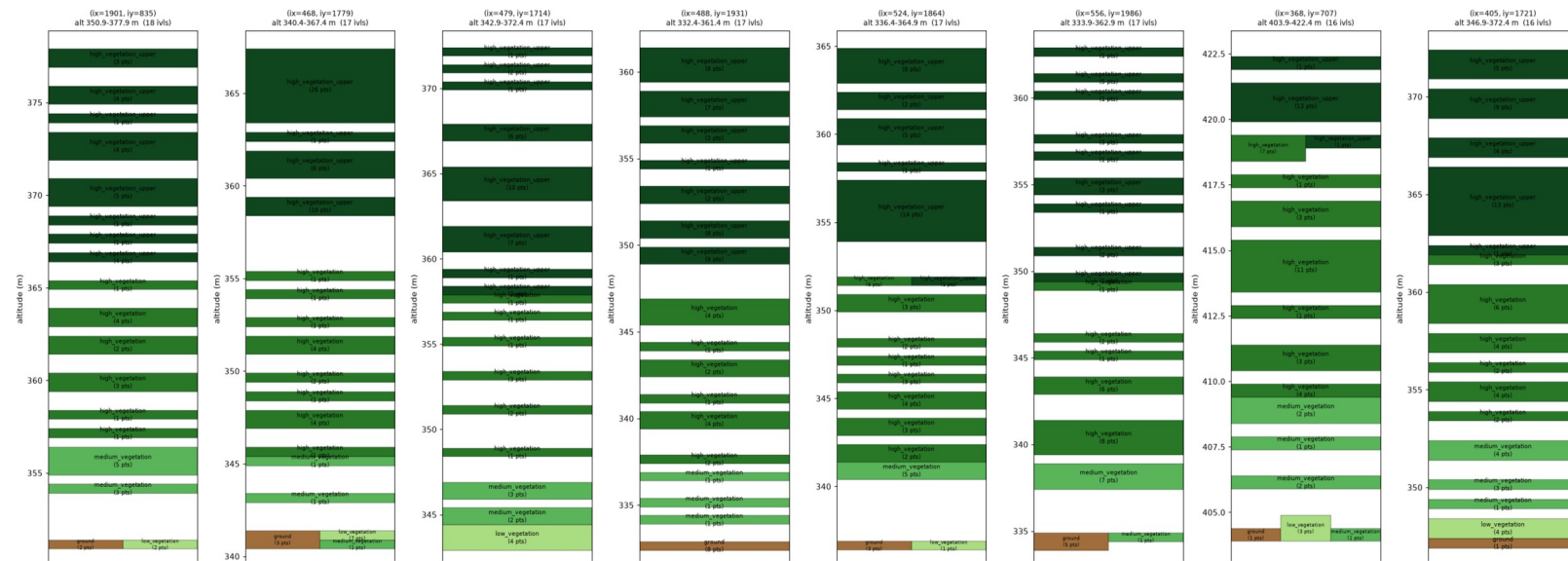
la mémoire est bornée avant de lancer ;
une défaillance ne coûte qu'un produit, et se reprend

aucune étape n'a besoin de la zone entière en mémoire

Annexe : regrouper les plages, la fragmentation

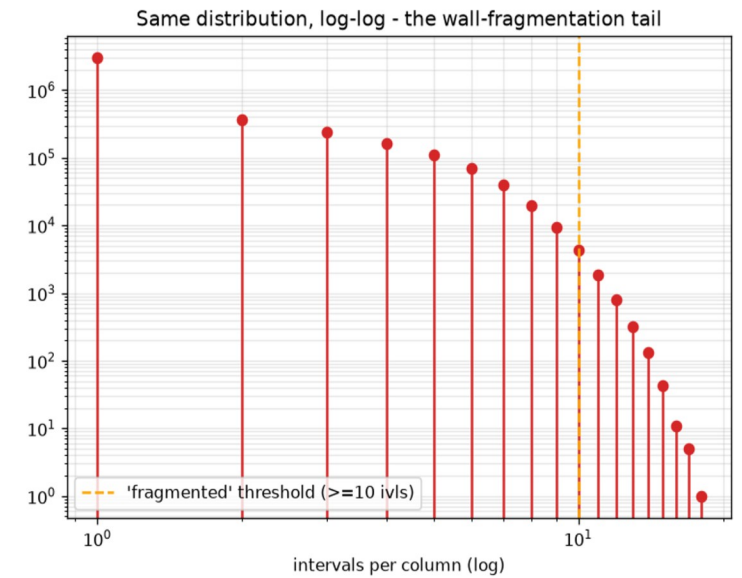
annexe

les colonnes les plus fragmentées du carré (avant regroupement)



jusqu'à 18 plages ; médiane 1 ; moyenne 1,66 ; la queue à droite est le coût de la fragmentation

plages par colonne, échelle log-log



458 583 colonnes sur 3 995 579 ont des plages à fusionner (écart $\leq 1,5$ m)

Annexe : regrouper les plages, la règle

annexe

écart $\leq 1,5$ m, même classe

avant



après



regroupées : une plage

écart $> 1,5$ m

avant



après



gardées séparées

classes différentes

avant



après



jamais fusionnées

tolérance en mètres : la même décision à toute résolution ; aucune classe ne change

1,5 m en production ; le stockage brut est conservé à côté du regroupé

Annexe : tuile par tuile, la preuve

Construction directe :
une seule passe, tout en mémoire
88 s

Construction partitionnée :
quatre fragments, fusion par bandes
95 s

Six tableaux comparés élément par élément :
0 différence
0 colonne divergente sur 3 995 579

Le même résultat :
3 995 579 colonnes, 5 830 031 pages,
52 705 942 points

construire tuile par tuile ne change rien au résultat : seulement la mémoire occupée

Annexe : les encodeurs, un post-traitement mesuré

quatre encodeurs appliqués après la construction, mesurés contre le stockage brut

encodeur v1 à v4	règle ce que l'encodeur change	réduction 1 m plages en moins	réduction 0,5 m plages en moins	Jaccard végétation contre le brut, 1 m	lecture ce que dit la mesure
v1	lissage sur 3 cellules	14 à 29 %	6 à 13 %	0,87 à 0,93	empreinte peu changée
v2	comblement, écart ≤ 1 m	10 à 20 %	4 à 8 %	1,00	empreinte inchangée
v3	classes mineures retirées	0 à 4 %	≈ 0 %	0,97 à 1,00	sans effet
v4	majorité au-dessus de 5 m	-2 à 5 %	-1 à 3 %	0,56 à 0,78	canopée déplacée

Jaccard : colonnes dont le sommet est végétation, communes aux deux stockages / présentes dans l'un ou l'autre ; 1,00 = même empreinte dix carrés d'un kilomètre, quatre résolutions ; v4 : J végétation 0,731 à 1 m sur la campagne du rapport

le regroupement de production n'est pas un encodeur : il ne change aucune classe

aucun encodeur n'est dans la chaîne livrée : l'analyse lit le brut ou le regroupé

Annexe : description des idées abandonnées

La grille dense :

de l'ordre du téraoctet, même à un octet par voxel ;
jamais construite

Le dictionnaire de colonnes :

874 octets de surcoût par colonne ;
effondré sur toute zone

Voter à la construction :

le feuillage disparaissait ;
les deux classes survivent, on tranche après

L'encodeur majoritaire :

27 % de la canopée en désaccord ;
rejeté pour l'analyse, gardé comme preuve

chaque abandon est caractérisé et conservé dans le dépôt

Annexe : idées abandonnées, en images (1)

la grille dense



3 × 3 km à 0,5 m : $\sim 10^{10}$ voxels, presque tous de l'air

la Métropole : $\sim 1,5$ To d'un coup ; jamais construite

le dictionnaire de colonnes

un objet Python par colonne

six tableaux plats



par colonne

16 octets

≈ 3 Go de surcoût par tuile fine ; mémoire épuisée sur toute zone

abandonné pour la structure de tableaux : 16 fois moins, tout vectorisé

stocker l'air, ou payer un objet par colonne : les deux tuent l'échelle

Annexe : idées abandonnées, en images (2)

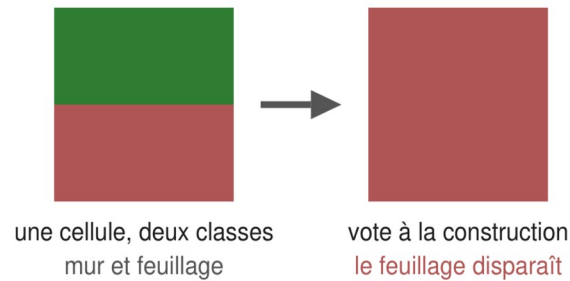
annexe

les confettis



18 plages pour un arbre : des retours sur les feuilles et les branches
mesuré d'abord (histogrammes), puis regroupé, jamais deviné

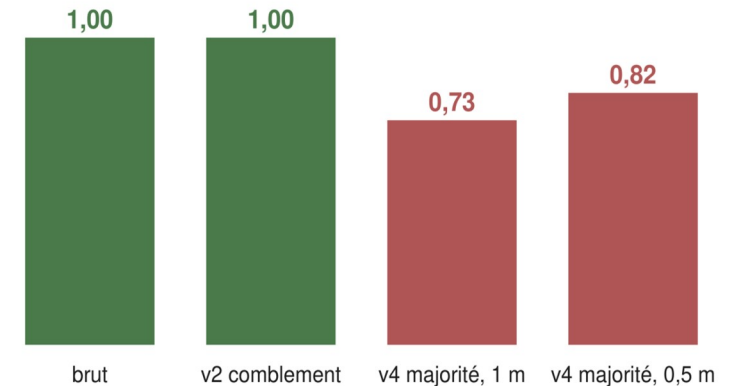
qui gagne le voxel ?



choix retenu : la construction garde les deux plages ;
un passage de résolution, séparé et mesuré, tranche si besoin

mesurer d'abord, interpréter ensuite

l'encodeur majoritaire (v4)



indice de Jaccard de l'empreinte végétation contre le brut

**27 % de l'union en désaccord à 1 m :
rejeté pour l'analyse, gardé comme preuve**

mesurer d'abord, simplifier ensuite, et garder la preuve du rejet

Run1, Run2 (2 m) :

689 s et 708 s ; 18,3 et 18,7 Go de pic ;
162,6 M colonnes ; 422,2 M / 426,9 M plages

Run3, Run4 (1 m) :

3 637 s et 2 943 s ; 26,6 et 27,1 Go ;
648,5 M colonnes ; 1 351,6 M / 1 453,4 M plages

Run5, Run6 (0,5 m) :

2 584,3 M colonnes ; 4 591,6 M / 5 418,4 M plages ;
24 et 304 Go de sorties

Le corpus :

2 842 tuiles, 51 912 212 206 points,
boîte 30 km × 44 km, EPSG:3946

chaque paire : regroupé puis non regroupé, mêmes 2 842 fragments

six tableaux plats, disposition CSR

par colonne : 16 octets

keys (uint64) : la clé de la colonne,
ix et iy dans ses deux moitiés

off (int64) : où commencent ses plages
colonne 977 169 : plages 1 552 851 à 1 552 854

par plage : 13 octets

zs, ze (int32) : début, fin exclue
cl (uint8) : la classe
ct (int32) : le nombre d'échos

plus un petit en-tête : origine et taille de cellule

la colonne (ix 488, iy 1931) après regroupement

zs	ze	classe	échos	altitude
14	15	2 (sol)	8	332,4 à 332,9 m
17	23	4 (vég. moyenne)	3	333,9 à 336,9 m
24	43	5 (vég. haute)	14	337,4 à 346,9 m
47	72	8 (vég. haute sup.)	37	348,9 à 361,4 m

+ 1 : l'indice de sol, calculé à la lecture (sommet de la plage de sol), jamais stocké

six tableaux décrivent ce qui est plein ; le septième nombre dit ce que vaut le vide

Air mesuré (-1) :

la colonne a un écho de sol
et le voxel est au-dessus de l'indice de sol

Intérieur opaque (-2) :

la colonne n'a pas d'écho de sol
et le voxel est sous son plus haut retour

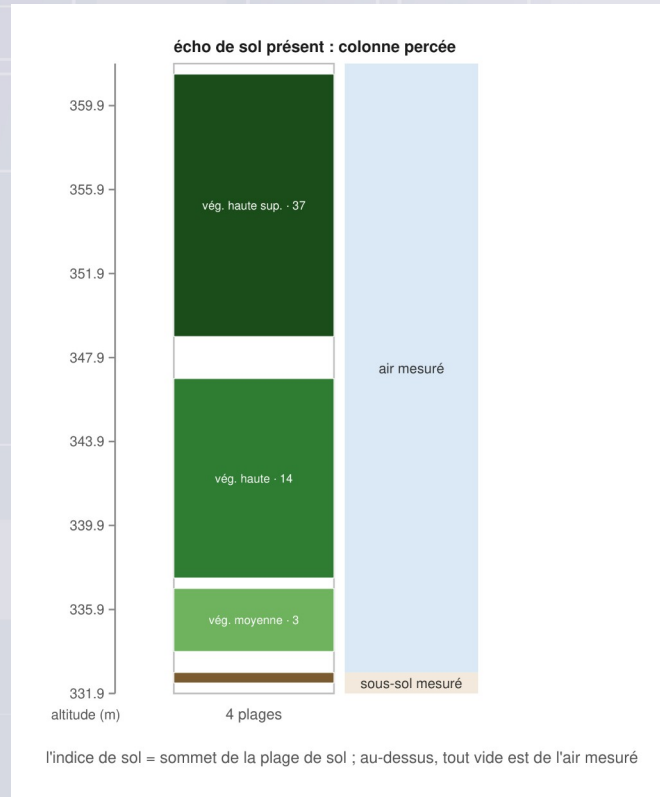
Sous-sol (-3) :

le voxel est sous l'indice de sol,
terre mesurée par l'écho

Au-dessus du plus haut retour :

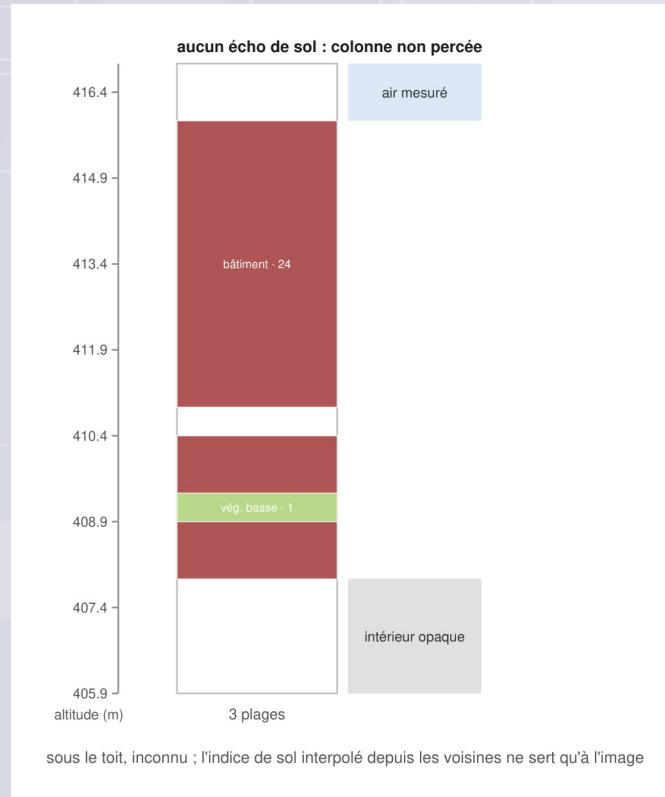
toujours de l'air mesuré :
le faisceau est descendu par là

le décodeur n'interpole jamais ; l'interpolation ne sert qu'aux images



La règle :
un écho de sol : la colonne est percée ;
sous l'indice de sol, de la terre mesurée ;
entre les plages et au-dessus, de l'air que le faisceau a
traversé

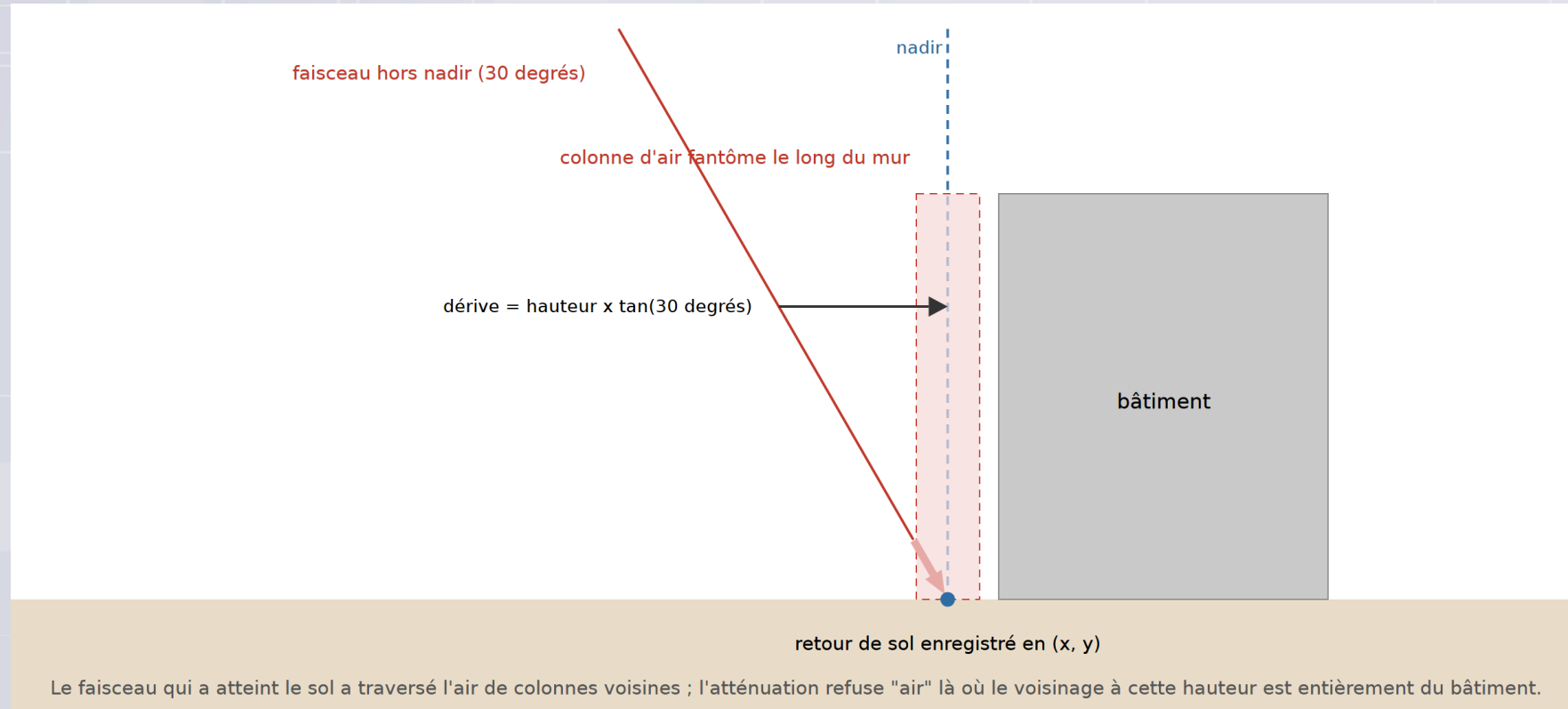
un seul nombre par colonne qualifie tout le vide : l'indice de sol



La règle :
aucun écho de sol : la colonne n'est pas percée ;
sous le toit, intérieur opaque, inconnu ;
au-dessus, de l'air mesuré ;
l'indice interpolé depuis les voisines ne sert qu'aux
images

le décodeur reste conservateur : ce qui n'a pas été mesuré est traité comme plein

Annexe : le faisceau hors nadir et la colonne fantôme



angles de balayage jusqu'à 30° : l'air traversé est attribué à une colonne voisine ; l'atténuation de voisinage refuse « air » là où le voisinage est bâti (rapport, figure 9)

Annexe : comment les visualiseurs fonctionnent

deux chemins depuis la même structure

visualiseur en flux (three.js)

stockage projeté
colonnes triées (ix, iy)

.bin : 32 octets / plage
position, taille, couleur

index par tuile de 100 m
boîte, décalage, nombre

score et budget GPU
4 M de boîtes par défaut

HTTP Range : le navigateur ne charge que les tuiles visibles

3D Tiles 1.1 (CesiumJS, iTowns, Unreal)

feuilles de 100 m
ordre de Morton

un niveau : pas doublé
plages fusionnées 2 à 2

nœud écrit puis libéré
node_i_j.glb instancié

tileset.json : EPSG 3946
vers ECEF ; 0 erreur

la pyramide se construit en flux : un niveau n'attend jamais le suivant en mémoire

35 images par seconde mesurées sur le campus ; validateur 3D Tiles : zéro erreur

Annexe : visualiseurs, mode d'emploi

Visualiseur en flux :

view_stream.cmd du niveau voulu ;
recherche par colonne, plage ou tuile, « Dim others »

CesiumJS :

serve_run.py puis viewer.html ;
seuil d'erreur 48 sûr jusqu'à 1080 px

iTowns :

3dtiles_loader.html sur le tileset local ;
sseThreshold 8, survol seulement

Unreal Engine :

Cesium for Unreal, maillages physiques désactivés ;
captures, jamais en direct

la même structure derrière chaque vue